SELECTED WORKS / AI·서비스

LangGraph 개인 AI 비서

검색 근거와 재질의 판단을 그래프로 드러내다

문서 검색·답변·재질의를 LangGraph 상태 그래프로 구성하고 배치 평가 도구를 갖춘 개인 AI 비서 프로젝트입니다.

https://github.com/MaoEmong/LangGraph-Rag

웹 상세 소개 보기

Python · LangGraph · FastAPI · Chroma · SQLite

핵심 흐름	분류 → 검색 → 생성 → 재질의 판단
검색 저장소	Chroma + SQLite FTS5
인터페이스	FastAPI · 웹 UI · CLI
평가 근거	평가 코드 공개 / 결과 산출물 미포함
DB 기능	QuerySpec·MySQL 어댑터 준비 단계

01 / CONTEXT

어떤 문제에서 시작했나요?

개인 문서를 검색해 답하는 시스템에서는 질문과 표현이 다른 문서를 찾는 능력, 정확한 단어를 놓치지 않는 능력, 답변에 필요한 문맥을 함께 모으는 능력이 필요합니다. 한 번의 벡터 검색과 생성만으로는 어떤 단계에서 근거가 부족해졌는지 드러나지 않습니다. 이 프로젝트는 검색·답변을 단계별 상태와 판단으로 나누는 데 초점을 두었습니다.

검색 전략을 더하면 품질뿐 아니라 지연과 운영 복잡도도 늘어납니다. 하이브리드 검색, 부모 청크 확장, 선택적 리랭커, 제한된 재질의를 각각 조절하고 평가할 수 있도록 구성했습니다. 응답 성공 여부와 키워드·인용 확인, 그래프 처리 시간을 나누어 읽는 도구도 함께 마련했습니다.

02 / PRODUCT FLOW

기능과 사용자 흐름

문서와 대화

파일 인제스트·질의응답·대화 스레드 조회와 초기화를 제공합니다.

검색과 재질의

Chroma·FTS5 하이브리드 검색, 부모 청크 확장과 제한된 재검색을 구성합니다.

평가 실행 도구

배치 실행·로그·실패 요약·리포트 분석 코드를 제공합니다. 과거 평가 결과 원본은 공개 트리에 없습니다.

한 번의 사용을 따라가 보면

01 문서 적재

파일이나 디렉터리를 인제스트하고 문서를 청크·메타데이터·벡터로 저장합니다. OCR은 이미지 자체를 이해하는 멀티모달 검색이 아니라 텍스트 추출을 통한 검색 경로입니다.

02 스레드를 지정해 질문

사용자가 thread_id와 질문을 보내면 그래프가 검색 필요 여부를 판단합니다. 스레드 조회·초기화 API를 통해 개인 대화 흐름을 관리합니다.

03 검색 자료로 답변 구성

필요한 경우 Dense·Sparse 검색과 부모 청크 확장을 수행하고 자료를 생성 단계에 전달합니다. 선택적 리랭커가 꺼진 설정과 켜진 설정을 구분해 실험할 수 있습니다.

04 근거가 부족하면 제한적으로 재검색

문서와 인용 수가 설정 기준에 못 미치면 질문을 검색용 키워드로 다시 쓰고 검색 단계로 돌아갑니다. 시도 횟수 상한에 도달하면 finalize로 진행해 무한 반복을 막습니다.

05 평가를 배치로 실행

동일한 평가 케이스를 여러 배치로 나누어 실행하고 결과와 실패 요약을 저장합니다. 그래프 내부 시간과 클라이언트에서 측정한 시간을 별도로 집계하도록 구성했습니다.

03 / ARCHITECTURE

구조와 책임을 나눈 방식

01 실제 StateGraph의 조건 분기

graph.py가 route·retrieve·db_plan·db_query·generate·requery·finalize를 LangGraph 노드로 등록합니다. 검색 필요 여부와 재질의 필요 여부로 분기하며 각 노드와 전체 실행 시간을 응답의 timing에 반영합니다. DB 노드가 그래프에 있다는 사실과 실제 DB 실연동 검증은 구분합니다.

02 문서·벡터·대화의 저장 분리

문서 정보와 대화 체크포인트는 SQLite, 벡터는 영속 Chroma에 둡니다. 인제스트는 source_path와 content_hash로 동일 파일을 건너뛰고 변경 파일을 교체하는 정책을 README에 명시합니다. 이미지 입력은 OCR로 추출한 텍스트를 검색 대상으로 사용합니다.

03 하이브리드 검색과 문맥 확장

Dense 검색과 SQLite FTS5 검색을 각각 수행해 chunk_id로 합치고 순위의 역수를 더해 정렬합니다. parent_id가 같은 청크를 확장해 문맥을 보충하며 리랭커는 off·auto·always 설정으로 선택합니다.

04 평가를 위한 별도 실행 도구

배치 러너가 서버 기동·health 확인·평가·종료를 반복하고 결과를 병합합니다. 분석기는 성공률·키워드·인용·지연 임계값과 장시간 검사 드리프트를 확인하고, 경고 시 실패 코드로 종료할 수 있습니다.

04 / ENGINEERING CASES

구현하면서 선택한 것

CASE 01 / 서로 다른 검색 점수는 순위로 결합

마주한 문제

임베딩 거리와 FTS5 점수는 같은 척도가 아니므로 원점수를 그대로 더하기 어렵습니다.

선택과 구현

각 검색 결과의 순위를 역수 점수로 바꾸고 chunk_id로 병합합니다. 검색 뒤 parent_id 기반 문맥 확장과 선택적 재정렬을 수행합니다.

남은 비용과 한계

순위 결합은 단순하고 추적하기 쉽지만 검색별 점수 차이의 크기를 버립니다. 부모 확장도 문맥을 늘리는 만큼 불필요한 자료와 토큰이 늘어날 수 있습니다.

retrieve 노드

CASE 02 / 재질의 판단을 별도 노드로 분리

마주한 문제

근거가 없는 검색을 계속 반복하거나 모든 질문에 재검색을 적용하면 지연과 API 사용량이 커집니다.

선택과 구현

검색 필요 여부, 문서·인용 최소 수와 attempt 상한을 확인합니다. 조건을 만족할 때만 질문을 다시 쓰고 retrieve로 돌아가며, 재작성 실패 시 원문을 유지합니다.

남은 비용과 한계

문서·인용 수는 근거의 의미적 정확성을 보장하지 않습니다. 재질의 상한은 지연을 통제하지만 어려운 질문을 충분히 탐색하지 못할 수 있습니다.

requery 노드

CASE 03 / 평가 실행의 실패를 산출물로 남김

마주한 문제

긴 평가 도중 서버가 불안정해지면 검색 품질 문제와 실행 환경 문제를 구분하기 어렵습니다.

선택과 구현

배치마다 서버를 재기동하고 health 확인 후 평가합니다. 배치별 로그·실패 단계·오류를 저장하고 성공·키워드·인용·지연 통계를 병합합니다.

남은 비용과 한계

재기동 배치는 연속 운영의 안정성 검사를 대신하지 않습니다. 또한 평가 코드의 존재는 통과 결과의 증거가 아니므로 실제 산출물과 환경을 함께 보관해야 합니다.

E2E 배치 러너

05 / ROLE & CONTRIBUTION

본인의 역할과 기여

이 프로젝트에서 소개하는 구현 범위는 FastAPI 인터페이스, 문서 인제스트·저장, LangGraph 상태 흐름, 검색·재질의와 평가 실행 도구입니다. 단순히 라이브러리를 연결했다는 설명보다 노드별 입력·분기·처리 시간과 설정의 역할을 소스로 따라갈 수 있도록 정리했습니다.

저장소는 개인용 AI 비서 프로젝트로 소개되어 있으며 별도의 팀 역할 표는 없습니다. 이 원고는 공개 코드로 확인 가능한 설계와 구현을 설명하며 협업 인원, 모든 커밋의 단독 작성 여부, 실측하지 않은 품질 개선치를 추가하지 않습니다.

06 / EVIDENCE & LIMITS

무엇을 확인했고, 무엇이 남았나요?

아래 결과는 연결된 공개 문서의 기록입니다. 이 소개를 갱신하면서 원본 프로젝트의 검사를 다시 실행한 결과는 아닙니다.

그래프 구현 근거

StateGraph 등록, route 분기, 재검색 루프와 노드별 timing 측정이 실제 코드에 있습니다.

구조 확인 결과이며 외부 모델을 연결해 전체 질의를 재실행한 검증은 아닙니다.

검증 근거 보기

평가 자동화 범위

분석기가 성공·키워드·인용·p90 지연의 임계값을 검사하고 fail-on-alert 옵션에서 종료 코드 2를 반환합니다.

평가 방법을 확인한 근거입니다. 임계값 충족이나 답변의 사실 정확도를 현재 증명하지 않습니다.

검증 근거 보기

과거 평가 주장과 공개 증거

README는 과거 50케이스의 성공·키워드·인용 통과 및 graph p90 약 6.7초를 보고합니다.

해당 결과·분석 JSON은 공개 트리에 없습니다. 원자료와 실행 환경을 대조할 수 없어 현재 성과나 재현된 수치로 사용하지 않습니다.

검증 근거 보기

현재 소개 범위와 한계

• README의 50케이스 결과 원본이 공개되지 않아 평가 통과 수치와 지연을 독립적으로 재검증할 수 없습니다.

• DB QuerySpec과 MySQL 어댑터는 준비 단계이며 실제 DB 연결은 환경 설정 뒤 스모크 검증이 필요합니다.

• 리랭커·OCR은 선택 기능으로 모델·GPU·Tesseract 등의 환경에 영향을 받습니다.

• 키워드·인용 개수 중심 검사는 답변의 사실성이나 문서의 공격성 입력에 대한 강건성 평가 전체를 대신하지 않습니다.

다음에 보완할 것

• 비밀정보를 제외한 평가 입력·결과·분석 JSON과 모델·설정·커밋을 함께 공개해 재현 가능하게 만들기

• 검색 실패·근거 불충분·재질의 상한 시나리오를 별도로 비교하고 DB 준비 기능의 실제 연결 결과를 분리 기록하기

07 / SOURCE MATERIAL

설명과 함께 확인할 근거

아래 공개 자료에서 구현 범위와 설계 판단을 확인할 수 있습니다. 링크를 누르면 원문으로 이동합니다.

01 현재 기능과 과거 평가 README

https://github.com/MaoEmong/LangGraph-Rag/blob/master/README.md

02 LangGraph 그래프

https://github.com/MaoEmong/LangGraph-Rag/blob/master/rag_assistant/app/graph.py

03 하이브리드 검색

https://github.com/MaoEmong/LangGraph-Rag/blob/master/rag_assistant/app/nodes/retrieve.py

04 재질의 판단

https://github.com/MaoEmong/LangGraph-Rag/blob/master/rag_assistant/app/nodes/requery.py

05 배치 평가 실행

https://github.com/MaoEmong/LangGraph-Rag/blob/master/rag_assistant/scripts/e2e_eval_batch_runner.py

06 평가 리포트 분석

https://github.com/MaoEmong/LangGraph-Rag/blob/master/rag_assistant/scripts/e2e_report_analyzer.py